

Cloud Load Balancer

Discussion Prep Sheet

Big Picture

Q: In one sentence, what does your project do?

A: It simulates a cloud load balancer that distributes incoming tasks across multiple virtual machines (servers) using Round Robin scheduling, with each server running as an independent thread.

Q: What OS concept does this demonstrate?

A: CPU scheduling — specifically Round Robin — applied to distributing work across server “resources” instead of a single CPU. It also demonstrates concurrency, producer-consumer synchronization, and resource allocation.

Scheduling Algorithm

Q: Why did you choose Round Robin instead of Shortest Job First or Priority Scheduling?

A: Round Robin is fair and simple — it guarantees every server gets tasks in strict rotation regardless of current load, so no server is starved. SJF would require knowing task burst times in advance to make routing decisions, which is less realistic for incoming requests whose processing time isn't known until they arrive. Priority Scheduling would need a defined priority system, which wasn't part of the assigned concept for this project.

Q: What's a downside of Round Robin here?

A: It doesn't account for current server load. If Server-1 is stuck processing a long task, it can still receive the next task in rotation and end up with a backlog, even if another server is idle. A load-aware algorithm (like Least Connections) would avoid that, but wasn't the assigned concept.

Q: How is the rotation actually implemented?

A: An AtomicInteger tracks the index of the “next” server. Each call to dispatch() does $(\text{index} + 1) \% \text{numberOfServers}$, which cycles through 0, 1, 2, ... and wraps back to 0 after the last server — that modulo operation is what creates the round-robin behavior.

Threading and Synchronization

Q: How many threads does your program use, and what does each one do?

A: One thread per server (each server continuously processes its own queue), one TaskGenerator thread producing new tasks, and the main thread that coordinates startup and shutdown.

Q: How do you avoid race conditions when multiple threads interact?

A: Two mechanisms: each server owns a BlockingQueue<Task>, which is thread-safe by design — the LoadBalancer (acting as producer) can safely add to it while the server thread (acting as consumer) safely removes from it, with no manual locking needed. Second, the LoadBalancer's rotation index is an

AtomicInteger, so even if `dispatch()` were called from multiple threads simultaneously, the index update is guaranteed atomic and correct.

Q: What is `BlockingQueue.take()` actually doing?

A: It blocks (pauses) the calling thread until an item is available in the queue. This means a server thread doesn't busy-wait or poll in a loop checking "is there a task yet?" — it just sleeps efficiently until one arrives, which is much closer to how a real OS scheduler avoids wasting CPU cycles on idle processes.

Q: Why use `Thread.sleep()` to simulate processing?

A: It's a simple way to represent a server being "busy" for a task's burst time without needing real computational work — the thread is occupied for that duration and can't pick up a new task until it finishes, which is exactly the behavior we want to simulate.

API Integration

Q: What does the geolocation API add to this project, functionally?

A: It satisfies the assignment's requirement to integrate a real external API. Each simulated task is given a sample IP address, and `GeoLocator` calls `ip-api.com` to resolve that IP to a real city/country, which then appears in the logs.

Q: Does the geolocation data affect which server a task goes to?

A: No — intentionally. Task routing is decided entirely by Round Robin. The geolocation lookup only adds descriptive information to each task for logging purposes, so it doesn't interfere with or complicate the scheduling algorithm being demonstrated.

Q: What happens if the API call fails?

A: `GeoLocator.lookup()` is wrapped in a try/catch block. Any failure — bad IP, timeout, no internet — is caught and the method returns "Unknown location" instead of throwing an exception and crashing the program. This was tested deliberately by including an invalid IP in the sample list, and it consistently returns the fallback string without affecting any other task.

Q: Why include a deliberately broken IP instead of only using valid ones?

A: To prove the error handling actually works, not just assume it does. It's evidence that a single failed external API call degrades gracefully rather than taking down the whole simulation — an important property for any real system that depends on external services.

Design and Extensibility

Q: How would you add a fourth server, or more tasks?

A: Both are just variables in main — `numServers` and the task count passed to `TaskGenerator`. Nothing else in the code is hardcoded to a specific number, since the server list and the round-robin modulo logic both scale automatically.

Q: How do you know Round Robin distributed tasks fairly?

A: The end-of-run summary tracks each server's completed task count and total busy time. In a typical run with 10 tasks across 3 servers, the split comes out to 4/3/3 — as even as possible given 10 isn't divisible by 3, which confirms the rotation is working correctly rather than favoring any one server.

Q: What would you change if you had more time?

A: Possibly a load-aware algorithm as a second selectable strategy for comparison, or a small GUI showing live server queue depths instead of console output — neither was necessary to demonstrate the core OS concept, so they were left out to keep scope manageable given the deadline.

Quick Reference: Class Responsibilities

Class	Responsibility
Task	Represents one request — ID, burst time, arrival time, origin location
Server	A virtual machine thread; owns a queue, processes tasks, tracks stats
LoadBalancer	Implements Round Robin; decides which server gets each task
TaskGenerator	Produces tasks on its own thread at randomized intervals
GeoLocator	Calls the external API to resolve an IP to a location
CloudLoadBalancer (Main)	Starts everything, coordinates shutdown, prints summary